

MICROSERVICES COURSE

Spring Boot & Spring Cloud — Professional Edition

Session 22

Advanced Patterns: Outbox, Idempotency & API Versioning

Closing Technical Debt from Sessions 4, 7, and 8 • Architecture Clinic #2 Preparation

Dr. ElSayed Mohamed Elsayed Baladoh

Tech Lead · Ph.D. · sayedbaladoh@yahoo.com

MICROSERVICES COURSE

Spring Boot & Spring Cloud — Professional Edition

Session 22 — Advanced Patterns

Outbox • Idempotency • API Versioning

Closing Technical Debt from Sessions 4, 7, and 8 • Architecture Clinic #2 Preparation

1. Session Overview

In the earlier sessions, our platform learned how to build microservices, communicate through Kafka, protect calls with retries/circuit breakers, and expose services through an API Gateway.

Those patterns are useful, but they also introduced three important technical-debt questions:

1. **What if the database transaction succeeds but the Kafka event is never published?**
 - This is the **dual-write problem**.
 - We solve it with the **Transactional Outbox Pattern**.
2. **What if a payment request is retried after the first payment actually succeeded?**
 - The same business action could happen twice.
 - We solve this with **Idempotency Keys**.
3. **What happens when we need a breaking API change?**
 - Existing clients must not suddenly stop working.
 - We solve the migration problem with **API Versioning** and a deprecation window.

This session is deliberately practical. We will modify the platform rather than build a separate toy application.

2. Learning Objectives

By the end of this session, you should be able to:

Transactional Outbox

- Explain the dual-write problem.
- Explain why saving a business entity and publishing a Kafka event are not one atomic transaction.
- Implement an outbox table.
- Write the business entity and outbox event in the same local database transaction.
- Implement a polling publisher.
- Explain why Outbox normally provides **at-least-once delivery**, not exactly-once delivery.
- Explain why consumers still need idempotent processing.

Idempotency

- Explain the duplicate-payment problem caused by retries.
- Explain an idempotency key.
- Generate one key per logical operation.
- Reuse the same key across retry attempts.
- Store the original result on the server.
- Understand why a simple check-then-insert implementation can still have a concurrency race.
- Understand the role of a database uniqueness constraint.

API Versioning

- Explain why `/api/products` creates breaking-change risk.
- Implement URI-based versioning such as `/api/v1/products`.
- Update API Gateway routes.
- Maintain a temporary legacy route.
- Use `Deprecation` and `Sunset` response headers.
- Explain a practical migration/deprecation cycle.

Architecture Clinic #2

- Review the Technical Debt Register.
- Identify which earlier issues were fixed.
- Identify which issues were intentionally left unresolved.
- Explain architectural trade-offs rather than simply saying that one design is "best."

PART 1 — TRANSACTIONAL OUTBOX

3. Knowledge Check

Before coding, answer these questions.

Q1 — Dual Write

What happens if:

```
OrderService.save()
    ↓
PostgreSQL succeeds
    ↓
Application crashes
    ↓
kafkaTemplate.send()
never happens
```

Answer

The order may remain in `PENDING` state, but the `OrderPlaced` event was never published.

Therefore:

```
Order exists
    ↓
No OrderPlaced event
    ↓
Inventory does not reserve stock
    ↓
Saga does not continue
```

The problem exists because PostgreSQL and Kafka are separate systems and there is no single transaction covering both.

4. The Dual-Write Problem

Imagine the original Session 7 code:

```
orderRepository.save(order);
```

```
kafkaTemplate.send(  
    "order-events",  
    order.getId(),  
    eventJson  
);
```

It looks reasonable.

But these are two independent operations.

Failure window

```
1. Save Order  
   |  
   | SUCCESS  
   v  
2. Publish Kafka Event  
   |  
   | CRASH / NETWORK FAILURE  
   v  
   NEVER HAPPENS
```

The database now contains:

```
Order  
status = PENDING
```

but Kafka contains no corresponding event.

The Saga cannot know that the order was created.

5. What If We Reverse the Order?

A common question is:

Why not publish to Kafka first and save the order afterward?

That simply reverses the problem.

Kafka publish

|

| SUCCESS

v

Application crashes

|

v

Database save never happens

Now consumers may receive:

OrderPlaced

for an order that does not exist in the database.

So:

DB → Kafka

and

Kafka → DB

both have a failure window.

The real problem is that we are trying to make two independent systems behave as though they share one transaction.

6. The Transactional Outbox Pattern

The key idea is:

Put the event into the same database transaction as the business change.

Instead of:

Database

+

Kafka

we do:

PostgreSQL Transaction

```
|
+-- orders
|
+-- outbox_events
```

Both database writes succeed or both roll back.

After the transaction commits:

outbox_events

```
|
v
Outbox Publisher
|
v
Kafka
```

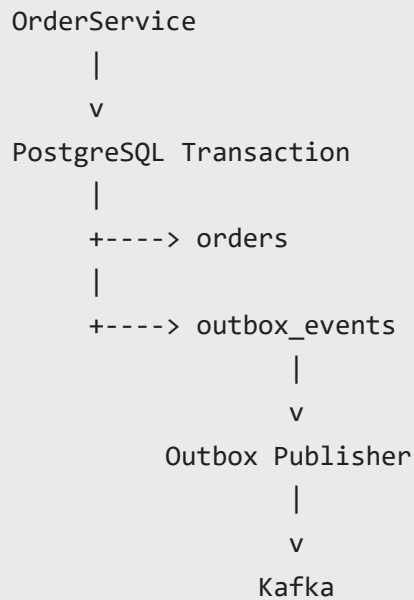
7. The New Architecture

Before

OrderService

```
|
+----> PostgreSQL
|
+----> Kafka
```

After



The application no longer directly publishes the business event from `createOrder()`.

8. Why This Works

Suppose the application crashes while creating an order.

Case A — Transaction never commits

orders	INSERT	ROLLBACK
outbox	INSERT	ROLLBACK

No incomplete state remains.

Case B — Transaction commits

Both records exist:

orders
+
outbox_events

If the application crashes immediately afterward, the outbox record is still there.

When the service starts again:

```
Outbox Publisher
  |
  v
find unpublished events
  |
  v
publish to Kafka
```

The event can still be delivered.

That is the major reliability improvement.

9. Polling Publisher vs CDC

There are two common ways to publish outbox events.

Option A — Polling

A scheduled task periodically queries:

```
WHERE published = false
```

Example:

```
Every 1 second
  |
  v
SELECT unpublished events
  |
  v
Publish to Kafka
```

Advantages

- Simple.

- Easy to understand.
- No additional infrastructure.
- Good for training and many moderate workloads.

Disadvantages

- Polling introduces some delay.
 - Repeated database queries create overhead.
 - Multiple service instances require careful coordination.
-

Option B — Change Data Capture

A CDC tool such as Debezium can read database changes from the database's transaction log/WAL and publish them downstream.

Conceptually:

```
PostgreSQL WAL
  |
  v
CDC / Debezium
  |
  v
Kafka
```

Advantages

- Low latency.
- Efficient for high-throughput systems.
- Avoids constant polling queries.

Disadvantages

- More infrastructure.
- More operational complexity.
- More components to monitor and maintain.

Our Course Decision

For this training platform:

Polling Publisher

is the selected implementation because it teaches the Outbox pattern without introducing another infrastructure layer.

10. Build the Outbox Table

Create:

```
@Entity
@Table(name = "outbox_events")
public class OutboxEvent {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String aggregateType;

    private String aggregateId;

    private String eventType;

    @Column(columnDefinition = "TEXT")
    private String payload;

    private boolean published = false;

    private Instant createdAt = Instant.now();

    // getters and setters
}
```

Meaning of the fields

Field	Meaning
<code>id</code>	Unique outbox record
<code>aggregateType</code>	Business object type, e.g. <code>Order</code>
<code>aggregateId</code>	ID of the order
<code>eventType</code>	e.g. <code>OrderPlaced</code>
<code>payload</code>	Serialized event
<code>published</code>	Whether the publisher has successfully published it
<code>createdAt</code>	Creation time

11. Repository

Create:

```
public interface OutboxEventRepository
    extends JpaRepository<OutboxEvent, Long> {

    List<OutboxEvent> findTop100ByPublishedFalseOrderByCreatedAtAsc();
}
```

We use a batch limit rather than loading every unpublished event.

For a production system, the repository would normally become more sophisticated as throughput and concurrency increase.

12. Write Order and Outbox in One Transaction

The important change is in `OrderService`.

```
@Service
public class OrderService {

    private final OrderRepository orderRepository;
    private final OutboxEventRepository outboxRepository;
    private final ObjectMapper objectMapper;

    public OrderService(
        OrderRepository orderRepository,
        OutboxEventRepository outboxRepository,
        ObjectMapper objectMapper) {

        this.orderRepository = orderRepository;
        this.outboxRepository = outboxRepository;
        this.objectMapper = objectMapper;
    }

    @Transactional
    public OrderResponse createOrder(OrderRequest request) {

        Order order = new Order(
            UUID.randomUUID().toString(),
            request.getProductId(),
            request.getQuantity(),
            request.getAmount(),
            OrderStatus.PENDING
        );

        orderRepository.save(order);

        OrderPlacedEvent event = new OrderPlacedEvent(
            order.getOrderId(),
            request.getProductId(),
            request.getQuantity(),
            request.getAmount(),
            request.getCustomerId()
        );
    }
}
```

```

    OutboxEvent outboxEvent = new OutboxEvent();

    outboxEvent.setAggregateType("Order");
    outboxEvent.setAggregateId(order.getOrderid());
    outboxEvent.setEventType("OrderPlaced");
    outboxEvent.setPayload(toJson(event));

    outboxRepository.save(outboxEvent);

    return new OrderResponse(
        order.getOrderid(),
        OrderStatus.PENDING,
        "Order received - processing..."
    );
}

private String toJson(Object object) {
    try {
        return objectMapper.writeValueAsString(object);
    } catch (JsonProcessingException e) {
        throw new IllegalStateException(
            "Could not serialize event", e);
    }
}
}

```

Important

There is no:

```
kafkaTemplate.send(...)
```

inside `createOrder()`.

The method has one responsibility:

Create business state

+

Record event to publish

Both are committed by PostgreSQL together.

13. What Does `@Transactional` Guarantee?

The transaction protects:

```
orderRepository.save(order)
```

and:

```
outboxRepository.save(outboxEvent)
```

as one local database transaction.

Conceptually:

```
BEGIN TRANSACTION
```

```
INSERT order
```

```
INSERT outbox_event
```

```
COMMIT
```

If an exception occurs:

```
ROLLBACK
```

Both database changes are rolled back.

Important Limitation

`@Transactional` does NOT create a transaction across PostgreSQL and Kafka.

It only makes the two PostgreSQL operations atomic.

Kafka remains outside the database transaction.

That is exactly why the publisher is a separate step.

14. The Outbox Publisher

Create:

```
@Component
public class OutboxPublisher {

    private final OutboxEventRepository outboxRepository;
    private final KafkaTemplate<String, String> kafkaTemplate;

    public OutboxPublisher(
        OutboxEventRepository outboxRepository,
        KafkaTemplate<String, String> kafkaTemplate) {

        this.outboxRepository = outboxRepository;
        this.kafkaTemplate = kafkaTemplate;
    }

    @Scheduled(fixedDelay = 1000)
    @Transactional
    public void publishPendingEvents() {

        List<OutboxEvent> pending =
            outboxRepository
                .findTop100ByPublishedFalseOrderByCreatedAtAsc();

        for (OutboxEvent event : pending) {

            String topic = switch (event.getEventType()) {
                case "OrderPlaced" -> "order-events";
                default -> throw new IllegalStateException(
                    "Unknown event type: " + event.getEventType());
            };
        }
    }
}
```

```

try {
    kafkaTemplate
        .send(
            topic,
            event.getAggregateId(),
            event.getPayload()
        )
        .get();

    event.setPublished(true);
    outboxRepository.save(event);

} catch (Exception e) {

    // Leave the event unpublished.
    // It will be retried by a later poll.
    throw new IllegalStateException(
        "Could not publish outbox event "
        + event.getId(), e);
}
}
}
}

```

Enable scheduling:

```

@Configuration
@EnableScheduling
public class SchedulingConfig {
}

```

15. Why Wait for Kafka Confirmation?

A subtle but important problem exists in the simple implementation:

```

kafkaTemplate.send(...);

```

```
event.setPublished(true);
```

`send()` is asynchronous.

If we mark the event as published immediately, we may mark it as complete before Kafka has actually acknowledged the message.

For this student implementation, we use:

```
.send(...).get();
```

This waits for the Kafka send result.

Only after successful acknowledgement do we execute:

```
event.setPublished(true);
```

This makes the example easier to reason about.

Production Note

Blocking on `.get()` is not necessarily the best high-throughput implementation.

Production implementations can use asynchronous callbacks/futures and more sophisticated batching, locking, retries, and status management.

For this course, correctness and clarity come first.

16. At-Least-Once Delivery

Outbox does NOT magically guarantee exactly-once delivery.

Consider:

- ```
1. Publish event to Kafka
 |
 | SUCCESS
 v
2. Application crashes
 |
```

v

3. published=true was never committed

After restart:

Outbox still says:  
published = false

The publisher sends the event again.

Therefore:

Outbox = AT-LEAST-ONCE delivery

not:

Exactly once

---

## 17. Why Duplicate Events Are Acceptable

This is where the consumer becomes important.

Suppose:

OrderPlaced  
OrderPlaced

are both received.

The consumer must not reserve inventory twice.

The consumer should use a unique event identity or business state check to make processing idempotent.

The overall architecture becomes:

Transactional Outbox  
|

```
 | at-least-once
 v
Kafka
 |
 v
Idempotent Consumer
```

This is a very common microservices reliability strategy.

---

## 18. Multi-Instance Warning

Imagine two Order Service pods:

```
Order Service #1
 |
 +----+
 |
Outbox Table |
 |
 +----+
 |
Order Service #2
```

Both instances might discover the same unpublished row.

That can result in duplicate publishing.

For a production-grade implementation, consider techniques such as:

- Row locking.
- `SELECT ... FOR UPDATE SKIP LOCKED`.
- Claiming/lease columns.
- Publisher ownership.
- CDC.
- Kafka producer configuration and consumer idempotency.

For this course, the important lesson is:

The pattern is correct, but the production implementation must also solve concurrent publisher coordination.

---

# 19. Demo — Prove the Pattern

## Step 1

Start:

```
PostgreSQL
Kafka
Order Service
Inventory Service
```

## Step 2

Create an order.

## Step 3

Immediately stop Order Service before the next polling cycle.

## Step 4

Inspect:

```
orders
outbox_events
```

You should see:

```
orders

orderId PENDING

outbox_events

id 1
eventType OrderPlaced
```

```
published false
```

## Step 5

Restart Order Service.

The scheduled publisher finds:

```
published = false
```

and publishes the event.

Then:

```
published = true
```

The Saga continues.

---

## 20. Outbox Failure Timeline

Without Outbox:

```
SAVE ORDER
 |
 X
CRASH
 |
KAFKA EVENT LOST
```

With Outbox:

```
SAVE ORDER
SAVE OUTBOX EVENT
 |
 v
COMMIT
```

```
 |
 X
 CRASH
 |
 v
 RESTART
 |
 v
PUBLISH OUTBOX EVENT
```

The difference is that the event is durably recorded before the application attempts to publish it.

---

## PART 2 — IDEMPOTENCY KEYS

### 21. The Duplicate Payment Problem

Recall Session 4.

Imagine:

```
Order Service
 |
 | Payment request
 v
Payment Service
 |
 | Charge succeeds
 v
Customer charged
```

But the response is lost:

```
Payment succeeds
 |
 X
Network timeout
 |
```

v  
Order Service sees failure

The retry mechanism does not know that the payment already succeeded.

It retries:

Attempt 2  
|  
v  
Payment Service  
|  
v  
Customer charged AGAIN

The customer can be charged twice.

---

## 22. What Is an Idempotency Key?

An idempotency key is a unique identifier for one logical operation.

Example:

Idempotency-Key:  
550e8400-e29b-41d4-a716-446655440000

The important rule is:

The same logical payment request must use the same idempotency key across retries.

Example:

Attempt 1  
Key = ABC-123  
|  
v  
Payment succeeds  
|

Response lost

Attempt 2

Key = ABC-123

|

v

Payment Service recognizes key

|

v

Return original result

The payment is NOT processed again.

---

## 23. The Key Belongs to the Logical Operation

Think carefully about:

HTTP attempt

versus:

Logical business operation

They are not the same.

One logical operation may have several HTTP attempts:

Logical Payment

|

--- Attempt 1

|

--- Attempt 2

|

--- Attempt 3

All three must use:

SAME IDEMPOTENCY KEY

## 24. Create the Idempotency Table

```
@Entity
@Table(name = "idempotency_records")
public class IdempotencyRecord {

 @Id
 private String idempotencyKey;

 @Column(columnDefinition = "TEXT")
 private String responseBody;

 private int statusCode;

 private Instant createdAt = Instant.now();

 // getters and setters
}
```

Repository:

```
public interface IdempotencyRecordRepository
 extends JpaRepository<IdempotencyRecord, String> {
}
```

The primary key gives us a database uniqueness guarantee.

## 25. Basic Idempotency Flow

The basic idea is:

Request  
|

```

 v
Read Idempotency-Key
 |
 v
Does key exist?
 / \
YES NO
 | |
 | v
 | Process payment
 | |
 | v
 | Store result
 | |
 v v
Return result

```

## 26. Payment Controller

A simple educational implementation:

```

@PostMapping
public ResponseEntity<PaymentResponse> processPayment(
 @RequestHeader("Idempotency-Key") String idempotencyKey,
 @RequestBody PaymentRequest request) {

 Optional<IdempotencyRecord> existing =
 idempotencyRepository.findById(idempotencyKey);

 if (existing.isPresent()) {

 IdempotencyRecord record = existing.get();

 return ResponseEntity
 .status(record.getStatusCode())
 .body(deserialize(
 record.getResponseBody(),

```

```

 PaymentResponse.class));
 }

 PaymentResponse response =
 paymentService.processPayment(request);

 IdempotencyRecord record =
 new IdempotencyRecord();

 record.setIdempotencyKey(idempotencyKey);
 record.setResponseBody(serialize(response));
 record.setStatusCode(200);

 idempotencyRepository.save(record);

 return ResponseEntity.ok(response);
}

```

The important behavior is:

```

if (existing.isPresent()) {
 return originalResult;
}

```

The second request does not charge the customer again.

---

## 27. The Critical Retry Rule

Do NOT do this:

```

@Retry(name = "paymentService")
public PaymentResponse callPayment(
 PaymentRequest request) {

 String key = UUID.randomUUID().toString();

 return paymentClient.processPayment(

```

```
 key,
 request);
 }
```

Why?

Because each retry may create a new key:

```
Attempt 1 → KEY-A
Attempt 2 → KEY-B
Attempt 3 → KEY-C
```

Payment Service sees three different requests.

The protection is defeated.

---

## 28. Correct Key Placement

Generate the key before the retry-wrapped operation:

```
String idempotencyKey =
 UUID.randomUUID().toString();

callPayment(request, idempotencyKey);
```

Then:

```
@Retry(name = "paymentService")
public PaymentResponse callPayment(
 PaymentRequest request,
 String idempotencyKey) {

 return paymentClient.processPayment(
 idempotencyKey,
 request);
}
```

Now:

Attempt 1 → KEY-A

Attempt 2 → KEY-A

Attempt 3 → KEY-A

Payment Service recognizes all retries as the same logical operation.

---

## 29. A More Important Production Detail: Request Mismatch

Suppose a client sends:

Idempotency-Key: ABC

amount = 100

Then later sends:

Idempotency-Key: ABC

amount = 500

Should the server return the original result?

Not blindly.

The same key should represent the same logical request.

A production implementation should store a request fingerprint/hash, for example:

key

request\_hash

response

status

created\_at

Then:

same key + same request

↓

```
return original result
```

```
same key + different request
```

↓

```
reject as misuse
```

This prevents accidental reuse of a key for a different payment.

---

## 30. Concurrency Race — Important Limitation

The basic implementation has another subtle issue.

Two requests can arrive at exactly the same time:

| Request A      | Request B      |
|----------------|----------------|
|                |                |
| v              | v              |
| find key       | find key       |
| NOT FOUND      | NOT FOUND      |
|                |                |
| v              | v              |
| charge payment | charge payment |
|                |                |
| v              | v              |
| save key       | save key       |

The customer could still be charged twice.

Therefore:

A simple `findById()` followed by processing is not sufficient for a fully production-grade implementation under concurrency.

The database uniqueness constraint helps detect duplicate insertion, but it does not automatically undo a payment that has already been performed.

For high-value payment systems, idempotency must be designed together with the payment provider's own idempotency mechanism and a robust state machine/transaction strategy.

---

## 31. Idempotency Is Not Only for Payments

The same idea applies to other operations:

- Creating an order.
- Submitting a payment.
- Sending a command.
- Processing a webhook.
- Creating a shipment.
- Charging a subscription.
- Handling duplicate Kafka messages.

Whenever:

```
same logical request
+
repeated delivery
```

must not create repeated side effects, idempotency is important.

---

## PART 3 — API VERSIONING

### 32. Why Version APIs?

Our old endpoint was:

```
/api/products
```

Suppose version 1 returns:

```
{
 "id": "P100",
 "name": "Laptop",
 "price": 1000
}
```

```
}
```

Later we need a breaking change:

```
{
 "productId": "P100",
 "displayName": "Laptop",
 "pricing": {
 "amount": 1000,
 "currency": "USD"
 }
}
```

Existing consumers may not understand the new response.

If we replace the old API directly:

```
/api/products
```

all consumers can break at once.

---

## 33. URI-Based Versioning

Our platform chooses:

```
/api/v1/products
```

and later:

```
/api/v2/products
```

Example:

```
Client A
 |
 v
/api/v1/products
```

```
|
v
Product Service

Client B
|
v
/api/v2/products
|
v
Product Service
```

---

## 34. Why Did We Choose URI Versioning?

For this course platform:

### Advantages

- Very visible.
- Easy to test in a browser or Postman.
- Easy to understand.
- Easy to route through Spring Cloud Gateway.
- Easy to see in logs.
- Works with ordinary HTTP clients.

### Trade-off

The URL changes between major versions:

```
/v1/
```

to:

```
/v2/
```

That is acceptable for our platform.

---

## 35. Other Versioning Strategies

URI versioning is not the only option.

### Header-Based

For example:

```
Accept: application/vnd.api.v1+json
```

Advantages:

- Version is separated from the URI.
- Can fit some API design approaches.

Disadvantages for our course:

- Less visible.
- More difficult to demonstrate.
- Less obvious in browser-based testing and logs.

### Query Parameter

For example:

```
/api/products?version=1
```

This is also possible, but it is not the strategy selected for our platform.

The important lesson is not:

URI versioning is always the best.

The lesson is:

Choose a strategy deliberately and document the trade-off.

---

## 36. Add `/api/v1`

Change:

```
@RestController
@RequestMapping("/api/products")
public class ProductController {
}
```

to:

```
@RestController
@RequestMapping("/api/v1/products")
public class ProductController {
}
```

The internal controller methods can remain unchanged if their contract has not changed.

---

## 37. Update API Gateway

The Gateway should now expose:

```
/api/v1/products/**
```

Example:

```
spring:
 cloud:
 gateway:
 routes:

 - id: product-service-v1
 uri: lb://PRODUCT-SERVICE
 predicates:
 - Path=/api/v1/products/**
```

The important point is that Gateway routing and service API contracts must evolve together.

---

## 38. Do Not Break Existing Clients Immediately

Imagine an existing client still calls:

```
/api/products
```

If we delete that route immediately:

```
Old Client
 |
 v
/api/products
 |
 x
404
```

Instead, we can temporarily maintain a compatibility route.

---

## 39. Legacy Redirect/Rewriting Route

Example:

```
spring:
 cloud:
 gateway:
 routes:

 - id: product-service-v1
 uri: lb://PRODUCT-SERVICE
 predicates:
 - Path=/api/v1/products/**

 - id: product-service-legacy-redirect
 uri: lb://PRODUCT-SERVICE
 predicates:
```

```
- Path=/api/products/**
filters:
- RewritePath=/api/products/(?<segment>.*),
/api/v1/products/${segment}
- AddResponseHeader=Deprecation, true
- AddResponseHeader=Sunset, "Wed, 01 Oct 2026 00:00:00 GMT"
```

The old request:

```
GET /api/products/P100
```

is internally rewritten to:

```
GET /api/v1/products/P100
```

---

## 40. Deprecation and Sunset

The compatibility response can communicate:

```
Deprecation: true
Sunset: Wed, 01 Oct 2026 00:00:00 GMT
```

Conceptually:

```
Old API
|
| still works
v
Deprecation Window
|
| clients migrate
v
New API
|
v
Legacy API removed
```

The important architectural idea is:

Do not surprise consumers with a breaking change when a controlled migration is possible.

---

## 41. One-Cycle Migration Strategy

Our platform can use:

### Phase 1 — Introduce v1

```
/api/v1/products
```

### Phase 2 — Keep legacy route

```
/api/products
 |
 v
/api/v1/products
```

Add:

```
Deprecation
Sunset
```

### Phase 3 — Migrate clients

Update:

- Gateway consumers.
- Tests.
- Internal services.
- Documentation.
- External integrations.

### Phase 4 — Remove legacy route

After the agreed deprecation cycle:

```
/api/products
```

is removed.

Only:

```
/api/v1/products
```

remains.

---

## PART 4 — INTEGRATING THE THREE PATTERNS

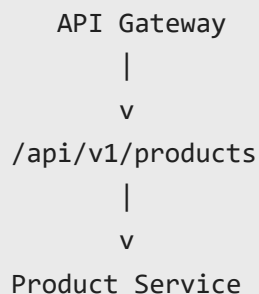
### 42. How the Patterns Fit Together

These are not three unrelated tricks.

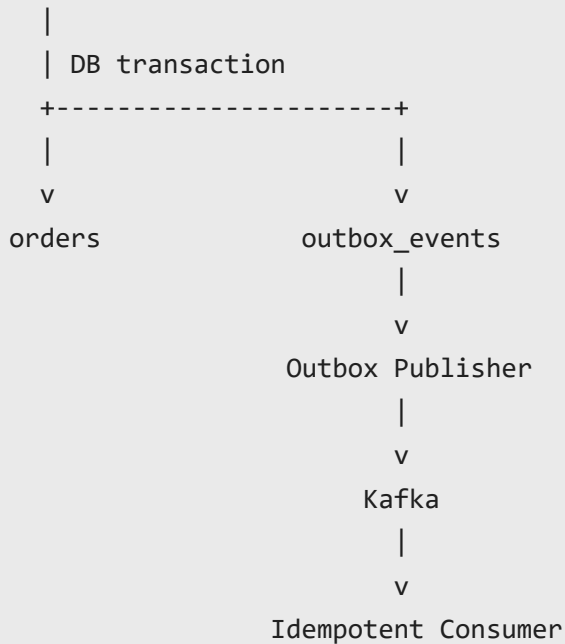
They solve three different distributed-system problems.

| Problem                                     | Pattern              |
|---------------------------------------------|----------------------|
| DB update + Kafka event can get out of sync | Transactional Outbox |
| Same request can execute multiple times     | Idempotency          |
| API contract can break consumers            | API Versioning       |

Together:



Order Service



Order Service



## 43. Important Distinction

Do not confuse:

### Transactional Outbox

with:

## Idempotency

Outbox answers:

How do I reliably record an event that must eventually be published?

Idempotency answers:

How do I prevent repeated processing of the same logical operation?

You often need both.

---

## 44. End-to-End Example

Consider creating an order.

### Step 1 — Client

```
POST /api/v1/orders
```

### Step 2 — Order Service

Inside one PostgreSQL transaction:

```
INSERT order
INSERT outbox event
COMMIT
```

### Step 3 — Outbox Publisher

```
outbox event
|
v
```

Kafka

## Step 4 — Payment Operation

The caller creates:

Idempotency-Key: PAYMENT-ABC-123

## Step 5 — Payment Service

First request:

```
key not found
 |
 v
process payment
 |
 v
store result
```

Retry:

```
key found
 |
 v
return stored result
```

This gives us stronger reliability at several different boundaries.

---

## PART 5 — LAB

### 45. Lab Goal

You will close three technical-debt items in the platform:

- [ ] Order Service uses Outbox
- [ ] Payment Service supports Idempotency-Key
- [ ] Product API uses /api/v1

---

## 46. Lab 1 — Implement Outbox

### Task 1

Create:

OutboxEvent

with:

id  
aggregateType  
aggregateId  
eventType  
payload  
published  
createdAt

### Task 2

Create:

OutboxEventRepository

### Task 3

Modify:

```
OrderService.createOrder()
```

so it:

1. Saves Order
2. Creates OrderPlaced event
3. Saves OutboxEvent

inside:

```
@Transactional
```

## Task 4

Remove the direct:

```
kafkaTemplate.send(...)
```

from `createOrder()`.

## Task 5

Create:

```
OutboxPublisher
```

using:

```
@Scheduled
```

## Task 6

Verify:

```
published = false
```

before publishing and:

```
published = true
```

after successful Kafka acknowledgement.

---

## 47. Lab 2 — Test Outbox Recovery

### Test

1. Start the platform.
2. Create an order.
3. Confirm the order exists.
4. Confirm the outbox row exists.
5. Stop Order Service before the next polling cycle.
6. Verify the row remains unpublished.
7. Restart Order Service.
8. Verify Kafka receives `OrderPlaced`.
9. Verify the Saga continues.

### Success Criteria

You have demonstrated that:

```
Application crash
```

```
|
```

```
v
```

```
Event is NOT lost
```

```
|
```

```
v
```

```
Restart
```

```
|
```

```
v
```

```
Outbox publisher retries
```

---

## 48. Lab 3 — Implement Idempotency

Create:

```
IdempotencyRecord
```

with:

```
idempotencyKey
responseBody
statusCode
createdAt
```

Modify the payment endpoint to require:

```
Idempotency-Key
```

Test:

### Request 1

```
Idempotency-Key: TEST-123
```

Expected:

```
Payment processed
```

### Request 2

Use exactly the same key:

```
Idempotency-Key: TEST-123
```

Expected:

Original payment result returned

The payment must not be processed again.

---

## 49. Lab 4 — Test Retry Behavior

Generate the key once:

```
String idempotencyKey =
 UUID.randomUUID().toString();
```

Then pass it to the retry-wrapped method.

Verify that all retry attempts use:

SAME KEY

Example:

```
Attempt 1 → KEY-123
Attempt 2 → KEY-123
Attempt 3 → KEY-123
```

---

## 50. Lab 5 — API Versioning

Change:

/api/products

to:

/api/v1/products

Update Gateway.

Then add a temporary legacy route:

```
/api/products/**
 |
 v
/api/v1/products/**
```

Add:

**Deprecation:** true

**Sunset:** Wed, 01 Oct 2026 00:00:00 GMT

Test both URLs.

---

## 51. Verification Checklist

Before finishing the session, verify:

### Outbox

- [ ] Order and OutboxEvent are written in one transaction
- [ ] Direct Kafka publish removed from createOrder()
- [ ] Publisher polls unpublished events
- [ ] Successful Kafka acknowledgement marks event published
- [ ] Failed publication leaves event available for retry
- [ ] You understand at-least-once delivery

### Idempotency

- [ ] Idempotency-Key is required
- [ ] First request performs the operation
- [ ] Retry with same key returns original result
- [ ] Key is generated outside the retry-wrapped method
- [ ] Database key is unique
- [ ] You understand the concurrent-request limitation

## API Versioning

- [ ] Product API exposes /api/v1/products
- [ ] Gateway has v1 route
- [ ] Legacy route works temporarily
- [ ] Deprecation header is returned
- [ ] Sunset date is communicated
- [ ] You understand the migration cycle

---

## PART 6 — COMMON MISTAKES

### 52. Mistake: Publishing Directly from the Transaction

Do not assume:

```
@Transactional
public void createOrder() {

 orderRepository.save(order);

 kafkaTemplate.send(...);
}
```

makes PostgreSQL and Kafka one transaction.

It does not.

The transaction is local to the database unless you deliberately configure a distributed/transactional messaging design.

The Outbox pattern avoids pretending that the two systems share one atomic transaction.

---

### 53. Mistake: Marking Outbox Published Too Early

Dangerous:

```
kafkaTemplate.send(...);
```

```
event.setPublished(true);
```

because `send()` is asynchronous.

For our educational implementation, wait for the Kafka acknowledgement before marking the record published.

---

## 54. Mistake: Assuming Outbox Means Exactly Once

Wrong:

```
Outbox = exactly once
```

Correct:

```
Outbox = reliable persistence + at-least-once publication
```

Consumers should be designed to tolerate duplicates.

---

## 55. Mistake: Creating a New Idempotency Key on Every Retry

Wrong:

```
Retry 1 → KEY-A
```

```
Retry 2 → KEY-B
```

```
Retry 3 → KEY-C
```

Correct:

```
Retry 1 → KEY-A
```

```
Retry 2 → KEY-A
```

Retry 3 → KEY-A

---

## 56. Mistake: Reusing a Key for Different Requests

Wrong:

KEY-A + payment \$100

KEY-A + payment \$500

The key identifies one logical operation.

Production implementations should validate the request fingerprint.

---

## 57. Mistake: Thinking a Unique Key Solves Everything

A database uniqueness constraint protects the record.

It does not automatically protect the external side effect.

For example:

```
Two requests arrive
 |
 v
Both see "not found"
 |
 v
Both charge payment
```

Therefore, serious payment systems require careful concurrency and provider-level idempotency design.

---

## 58. Mistake: Versioning Only the Controller

Changing:

```
@RequestMapping("/api/v1/products")
```

is not enough.

You must also consider:

- Gateway
- Tests
- Clients
- Documentation
- External consumers
- Monitoring

API versioning is an ecosystem concern, not just a controller annotation.

---

## PART 7 — ARCHITECTURE CLINIC #2

### PREPARATION

## 59. Why Are We Doing This Now?

Session 8 reviewed the platform after Phase 1.

Session 24 will review the platform after the larger journey.

The purpose is not to ask:

Did we implement every possible pattern?

Instead ask:

What technical debt did we identify, what did we fix, what did we deliberately leave, and why?

---

## 60. Technical Debt Review

Prepare a table like this:

| Debt / Decision                   | Originally Identified | Current State        | Decision              | Reason                   |
|-----------------------------------|-----------------------|----------------------|-----------------------|--------------------------|
| Direct DB + Kafka dual write      | Session 7             | Outbox               | Fixed                 | Reliability              |
| Payment retry without idempotency | Session 4             | Idempotency Key      | Fixed                 | Prevent duplicate charge |
| No API versioning                 | Session 8             | /api/v1              | Fixed                 | Controlled evolution     |
| Polling vs CDC                    | Session 22            | Polling              | Deliberately selected | Simplicity               |
| Exactly-once messaging            | Session 22            | Not assumed          | Deliberately left     | Complexity/trade-off     |
| Advanced outbox locking           | Session 22            | Basic implementation | Future improvement    | Course scope             |

## 61. Questions to Prepare for Session 24

Be ready to answer:

### Question 1

Which technical-debt item produced the greatest reliability improvement?

Explain why.

### Question 2

Why did we choose Outbox instead of directly publishing to Kafka?

### Question 3

Why did we choose polling instead of CDC?

## Question 4

Does Outbox guarantee exactly-once delivery?

Explain the failure window.

## Question 5

Why is consumer idempotency still necessary?

## Question 6

Why must the same idempotency key be reused across retries?

## Question 7

What happens if two requests with the same idempotency key arrive concurrently?

## Question 8

Why did we choose URI-based API versioning?

## Question 9

Why should a deprecated API not necessarily disappear immediately?

## Question 10

Which parts of our implementation are suitable for production, and which parts would require further hardening?

---

# 62. Architecture Clinic Mindset

There is no requirement that every design must be the most sophisticated possible design.

For example:

Polling

may be the correct decision for our current platform.

A larger organization may later choose:

```
CDC + Debezium + Kafka Connect
```

That does not mean the original decision was wrong.

Architecture is about:

```
Requirements
+
Constraints
+
Trade-offs
+
Operational reality
```

---

## 63. Session Summary

Today we closed three important technical-debt areas.

### Transactional Outbox

Instead of:

```
DB write
+
Kafka write
```

we use:

```
DB transaction
|
+-- business data
+-- outbox event
```

Then:

Outbox Publisher

|

v

Kafka

This gives durable event recording and at-least-once publication.

---

## Idempotency

Instead of allowing:

Retry

→ duplicate payment

we use:

Logical operation

|

v

Idempotency-Key

|

v

Same key on retries

|

v

Original result

---

## API Versioning

Instead of:

/api/products

we use:

```
/api/v1/products
```

while temporarily supporting:

```
/api/products
 |
 v
/api/v1/products
```

with a deprecation window.

---

## 64. Final Mental Model

Remember these three sentences:

### Outbox

**Record the event atomically with the business change, then publish it asynchronously.**

### Idempotency

**Give one logical operation one stable key, and reuse that key across retries.**

### API Versioning

**Change contracts deliberately and give consumers a migration path.**

These are not merely implementation techniques.

They are mechanisms for controlling failure, repetition, and change in distributed systems.

---

## 65. Final Assessment Questions

### Q1

What is the dual-write problem?

- A. Two Kafka consumers read the same message
- B. A database write and a Kafka publish are independent operations
- C. Two databases use different schemas
- D. A service has two controllers

**Answer: B**

---

## Q2

What does the Outbox pattern primarily guarantee?

- A. Exactly-once Kafka delivery
- B. Zero network failures
- C. Durable recording of events together with the business transaction
- D. No duplicate messages

**Answer: C**

---

## Q3

When should an outbox event normally be marked as successfully published?

- A. Immediately after calling `send()`
- B. Before sending to Kafka
- C. After successful Kafka acknowledgement
- D. When the application starts

**Answer: C**

---

## Q4

What should happen when the same idempotency key is received again?

- A. Always perform the operation again
- B. Return the original stored result
- C. Generate a new key
- D. Delete the old record

**Answer: B**

---

## Q5

Where should an idempotency key be generated when the call is retried?

- A. Inside every retry attempt
- B. Once before the retry-wrapped operation
- C. Inside Payment Service only
- D. After the payment succeeds

**Answer: B**

---

## Q6

What is the platform's chosen API versioning strategy?

- A. Query parameter
- B. Header only
- C. URI-based versioning
- D. No versioning

**Answer: C**

---

## Q7

Why keep a legacy API route temporarily?

- A. To increase latency
- B. To allow consumers time to migrate
- C. To remove testing
- D. To avoid documentation

**Answer: B**

---

## Q8

What delivery semantic does a basic polling Outbox provide?

- A. At-most-once
- B. Exactly-once
- C. At-least-once
- D. Never-once

**Answer: C**

---

## 66. Take-Home Challenge

Extend the implementation with the following improvements.

### Challenge 1 — Request Fingerprint

Add:

```
requestHash
```

to the idempotency record.

Behavior:

```
same key + same request
 → return original result
```

```
same key + different request
 → reject request
```

### Challenge 2 — Outbox Retry Information

Add fields such as:

```
attemptCount
lastAttemptAt
lastError
```

Then expose useful logs when an event repeatedly fails.

### Challenge 3 — Outbox Concurrency

Research how multiple service instances can safely claim outbox rows using:

```
SELECT ... FOR UPDATE SKIP LOCKED
```

Do not implement it blindly; explain the trade-off first.

## Challenge 4 — API v2

Design:

```
/api/v2/products
```

with one intentional breaking response change.

Keep:

```
/api/v1/products
```

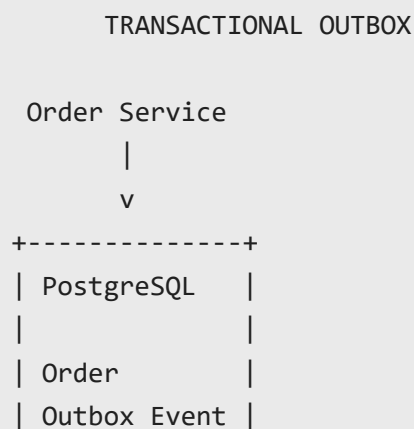
working at the same time.

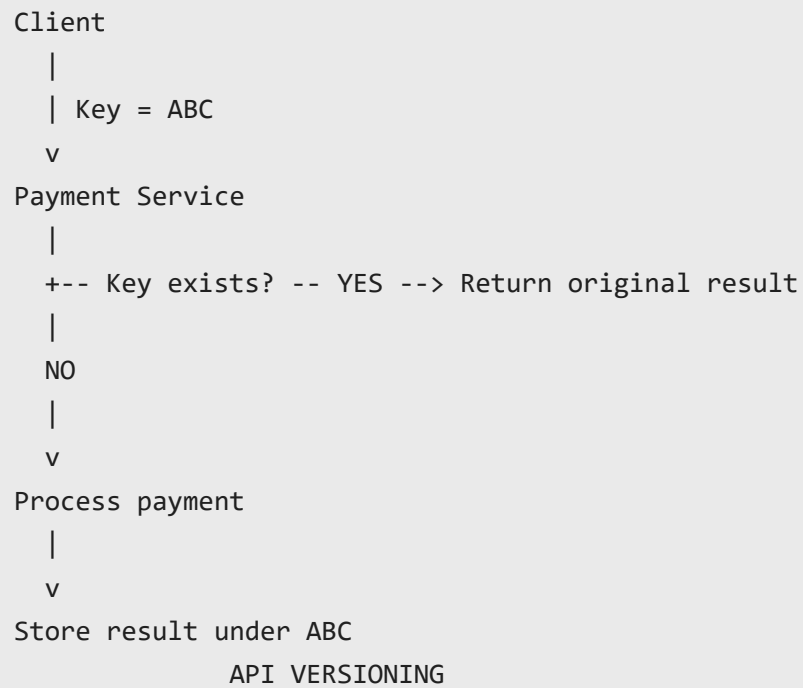
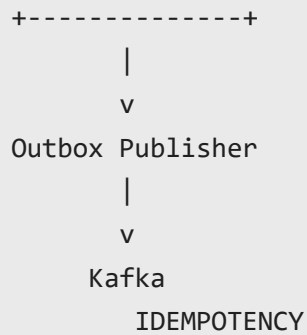
Explain how the Gateway routes both versions.

---

## 67. What You Should Be Able to Explain Without Code

At the end of this session, you should be able to explain these diagrams on a whiteboard:





```
New Client
 |
 v
/api/v1/products
```

If you can explain these three flows clearly, you understand the core of Session 22.

---

## 68. Final Takeaway

Distributed systems fail in predictable ways.

A database can succeed while Kafka fails.

A request can succeed while its response is lost.

An API can evolve while its consumers are still using the old contract.

The patterns from this session give us deliberate responses:

```
Database + Event
 ↓
 Outbox

Retry + Side Effect
 ↓
 Idempotency

Breaking API Change
 ↓
 Versioning + Migration
```

The professional engineering skill is not memorizing these patterns.

It is recognizing the failure mode, choosing an appropriate pattern, understanding its limitations, and being able to justify the trade-off.